EECS 348 Assignment 5: Two AIs Modify a Classmate's Sudoku Solver

Francis Tryban · EECS 348 · October 29, 2026

For this assignment I received Weston Uptmor's Sudoku solver and used two GenAIs to add solution files. Both got the same prompt and original code. Claude Opus 5.5 helped with the setup and model runs. OpenAI GPT-6.1-Sol in Zed helped test and compare the programs, revise the chosen code, and edit this analysis using my writing preferences. The two original AI programs below are unchanged.

2a–2b. The two GenAIs

I used **Claude Haiku 4.5** through Claude Code in my Mac's terminal and **GPT-6 Luna** through the Codex desktop app, set to Light effort. Each worked in a separate folder. These are the same models I used for Assignments 3 and 4, but this time they were modifying another student's code instead of starting from scratch.

The comparison and choice appear first. The exact prompt and both complete original programs follow in Appendices A and B.

2e. My comparison

To start my comparison I began by looking at the outputs. Both produce the correct solutions for the five provided puzzles, but in the wrong format. The solution grids need spaces between entries and a new line for each row, like the input files. Claude prints the entries smushed together with no spacing, while Codex reuses Weston's fancy printing with lines separating the 3-by-3 boxes.

Correctness

Both need a fix before they will compile with the plain course command. They keep a missing definition for `SudokuBoard::EMPTY` from Weston's code. They run with `-O2`, an optimization flag, but relying on that does not fix the problem. When run that way, both find the expected solutions for the five supplied puzzles and write the correct count or no-solution message. Their file formatting still needs to be fixed.

Claude preserves the original console output while adding solution files. Codex produces the files but removes the console output, so it does not fully preserve Assignment 4's required functionality.

Claude also keeps Weston's limit of 1,000 solutions, while Codex removes it. The limit does not affect the five provided puzzles, but it could leave out solutions on another puzzle. That goes against the requirement to find all solutions. Claude does not check whether its output file opens. Codex checks, but stops without telling the user what went wrong.

Execution time

Both keep Weston's recursive backtracking: try a digit, continue to the next empty cell, and undo the guess when returning. With k empty cells, a simple worst-case bound is O(9^k), since each cell has up to nine choices. Checking Sudoku rules cuts out many of those choices, but neither model changes how the next cell is chosen.

Claude does more printing because it writes to the console and a file. Codex writes only to the file, but removing required output is not a good way to improve the program. Both have the same basic search cost, and writing more solutions takes more time. Claude's cap can make it stop sooner, but that is unfinished work, not a better algorithm.

Space complexity

Both keep a fixed-size board and write each solution as soon as it is found. Neither stores a growing list of solved boards. The recursive calls use O(k) stack space, where k is the number of empty cells, and the board has only 81 cells. Both also load the input file into a string, so that memory depends on the file size. Claude adds another output stream, but it does not need another list of solutions. There is no major difference in how the two use memory.

Maintainability

Both keep Weston's board, solver and runner classes. Claude adds a separate file-printing method, which makes its spacing problem easy to locate without changing the console printer. Codex reuses the console printer for the file, so changing it would also affect how that printer could be used on screen.

Ignoring line endings, Git shows 38 added and 6 removed lines for Claude, compared with 23 added and 30 removed for Codex. Claude adds more but removes less of Weston's code. Both changed his Windows-style line endings, which makes the normal diff look much larger. Keeping his line endings matters for showing that the final file was modified, not replaced.

Both add a revision date but put the AI's name after `Revised by:` instead of mine. Their headers and comments need clearer credits. Claude's filename helper also searches for letters from the word `puzzle` instead of directly finding the number. Codex's approach of looking for digits is easier to follow. These are small parts that can be fixed without rewriting the solver.

2f. Which program I chose

I chose Claude's version because I think its printing bug is easier to fix. It already separates file printing from console printing, so I can add spaces without changing the original display. It also keeps more of Weston's existing code. Codex does get rid of the solution cap, but I would need to restore the console report as well as fix its file format. Claude's cap and missing file check still need attention, but neither requires replacing the solver.

2g. How I improved it

With GPT-6.1-Sol's help, I made these changes to the Claude-based version while keeping Weston's original structure:

- **Correctness:** Added spaces between entries in the file printer, set the default solution limit to zero so the search is unlimited, and added the missing definition of `EMPTY` so the plain compile command works. I also added messages for output-file failures.
- **Execution time:** Kept the existing backtracking search rather than adding a new algorithm. Removing the cap can make a run longer, but that is necessary to find all solutions. I am not claiming a speed improvement.
- **Space complexity:** Kept writing each solution immediately instead of storing solved boards. The changes do not add a growing collection in memory.
- **Maintainability:** Added my revision credit and clearer collaborator information, made the filename helper look for digits, and kept Weston's line endings so unrelated lines do not look rewritten.

The revised program compiled on cycle1 with `g++ Assignment5.cpp -o Assignment5.exe`, without extra flags, and ran on the five supplied puzzle files. It kept the console report and created all five solution files with spaced rows and the required final messages.

Appendix A — 2c. The prompt

I put the same instructions in `Claude/PROMPT.md` and `Codex/PROMPT.md` and checked that the files matched byte for byte. Each folder also had Weston's `Assignment4.cpp`, unchanged, and the five puzzle files. Both `AGENT.md` files were empty. Neither model saw the rubric, Weston's use case diagram, or the other model's code.

The prompt has three parts:

1. The modification requirements from the Assignment 5 instructions, word for word. I left out only the note about which GenAIs are allowed.
2. The Assignment 4 instructions, so the models knew what "required functionality from Assignment 4" means. I left out the line about `solution1.txt` being a debugging aid, because Assignment 5 doesn't provide that file and the program now writes its own.
3. A short note on where the files are.

I told both models:

```
Read AGENT.md and PROMPT.md. Write a modified version of Assignment4.cpp that satisfies PROMPT.md.
Save it to iterations/00-raw.cpp. Do not create or modify any other files.
```

Here is the full prompt both models received:

```
For this assignment, you will modify another student's Assignment 4.

Modify the Assignment 4 code you receive as follows:

- Write the solutions for each puzzle file to a file named solutionN.txt, where N is the puzzle
number.
- Each solutionN.txt file should include the following:
  - If no solutions are found, a single line: "No solution found."
  - If solutions are found, it should include all solutions followed by a single line: "Solutions
found: n", where n is the number of solutions found.
  - The solutions should be in the same format as the input files: puzzle1.txt – puzzle5.txt.
- You must modify the Assignment 4 you receive, not your Assignment 4.
- Do not replace the entire file with your code. Your GTA will use the Github Blame button to check
your modifications.
- Add your name to the Profile Comments as "Revised by:" along with a revision date.
- You must get all required functionality from Assignment 4 working, even if the student you inherited
 it from did not.

---

## Assignment 4 instructions

The program you received was written for these Assignment 4 instructions:

Write C++ code using objects, not functions, to implement the following requirements:

- Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
- A typical puzzle is shown here (image credit: Wikimedia Commons): [image not reproduced]
- Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following
constraints:
  - Each digit from 1 to 9 must appear exactly once in each row.
  - Each digit from 1 to 9 must appear exactly once in each column.
  - Each digit from 1 to 9 must appear exactly once in each block.
- Published puzzles are usually constructed to have a unique solution. This requires that at least 17
cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many
as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are
already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
- The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least
a) solution; if no solution exists, prove that.
```

- It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.
- That is not the approach we’re going to use. Instead, we’re going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
- Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).
- This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
- You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
- Write a recursive object-oriented program to find the solution to the five puzzle files.
- The output for each solution, should include:
  - The puzzle file name (e.g., puzzle1.txt)
  - The puzzle from the file printed out.
  - The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
- The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.

Input clarification

Assignment4.cpp, in the same folder as this file, is the Assignment 4 program
you received from another student. The five puzzle files (puzzle1.txt through
puzzle5.txt) are in the same folder.

The program must find its puzzle files in either of two ways:

- If no command-line arguments are given, solve puzzle1.txt, puzzle2.txt,
  puzzle3.txt, puzzle4.txt and puzzle5.txt, in that order, from the current
  working directory: `./program`
- If one or more filenames are given as command-line arguments, solve those
  files instead, in the order given: `./program puzzle3.txt`

Each solutionN.txt file is written to the current working directory. The
program exits with status 0 when it completes normally.

Appendix B — 2d. The programs they generated

Both programs are shown exactly as the models saved them, unedited. The copies are in my working folder as `Claude/iterations/00-raw.cpp` and `Codex/iterations/00-raw.cpp`.

Claude Haiku 4.5 (`Claude/iterations/00-raw.cpp`)

```
//EECS 348 Assignment 4
//Sudoku Solver that use a brute force recursive algorithm
//input: puzzle1.txt through puzzle5.txt
//Output: solved puzzles written to solutionN.txt files
//Collaborators: Me
//Other Collaborators: Claude
//Weston G. Uptmor
//10/15/2026
//Revised by: Claude (AI) 10/28/2026

#include <array> //used for grid
#include <cctype> //checks whether a character is a digit
#include <fstream> //for reading files
#include <iostream> //for output
#include <iterator> //reads a whole file as a string
#include <string> //allows the use of strings
#include <vector> //allows a list of filenames

class SudokuBoard { //initializes class SudokuBoard responsible for the board and the rules of placement
    public:
        static const int N = 9;      // board is N x N
        static const int B = 3;      // block is B x B
        static const int EMPTY = 0;  // internal marker for a blank square

        SudokuBoard() { grid .fill(EMPTY); } //constructor that starts with every cell blank
        //following code reads puzzle from a file and returns it
        //code returns false if the file is invalid
        bool load(const std::string& filename) { //method "load"
            std::ifstream in(filename); //opens file
            if (!in) return false; //if file cannot be opened return false
            std::string text((std::istreambuf_iterator<char>(in)), //read entire file into one string
                             std::istreambuf_iterator<char>());
            int count = 0; //initializes the count of cells that have been read to 0
            for (char c : text) { //for loop that looks at each character in the file
                if (count >= N * N) { //if grid has been fully read
                    if (std::isdigit(static_cast<unsigned char>(c)) || c == '_') //if there are too many
 cells
                        return false;  // return false
                    continue; //ignore whitespace and other characters
                }
                if (c >= '1' && c <= '9')            grid_[count++] = c - '0'; //if a digit is already
given, convert it to an integer and store it
                else if (c == '_' || c == '.' || c == '0') grid_[count++] = EMPTY; //otherwise store it
as a blank space
                // all newlines and spaces are skipped
            }
            return count == N * N; //success if exactly 81 cells were read
        }

        int  get(int row, int col) const { return grid [row * N + col]; } //read a cell
        void set(int row, int col, int v) { grid [row * N + col] = v; } //write a value into said cell
        bool isEmpty(int row, int col) const { return get(row, col) == EMPTY; } //return true if cell is
 blank

        bool canPlace(int row, int col, int v) const { //initializes method canPlace that checks if num
can be place in a given spot
            for (int i = 0; i < N; ++i) { //for loop that iterates along a row and a col together
                if (get(row, i) == v) return false;//if v already appears in this row, return false
                if (get(i, col) == v) return false;//if v already appears in this collumn, return false
```

```
            }
            int br = (row / B) * B, bc = (col / B) * B; //top left corner of a cell's 3x3 block
            for (int r = br; r < br + B; ++r) //iterates over block's rows
                for (int c = bc; c < bc + B; ++c) //iterates over block's column's
                    if (get(r, c) == v) return false; //if v already appears in this block, return false
            return true;//if placement is good, return True
        }

        bool givensAreConsistent() const { //initializes method "givensAreConsistent" which checks if
the puzzle is unsolvable
            SudokuBoard tmp = *this; //creates a copy of the current board
            for (int r = 0; r < N; ++r) //iterates over rows
                for (int c = 0; c < N; ++c) { //iterates over columns
                    int v = tmp.get(r, c); //digit in the current cell
                    if (v == EMPTY) continue; //if the current cell is a blank, go to next one
                    tmp.set(r, c, EMPTY); //blanks out current cell
                    bool ok = tmp.canPlace(r, c, v); //checks if it would be legal to put current cell
back
                    tmp.set(r, c, v); //puts the digit back
                    if (!ok) return false; //if there is a conflict, returns false
                }
            return true; //if there were no conflicts, returns true
    }

    void print(std::ostream& os) const { //initializes method "print" which prints out the board
        for (int r = 0; r < N; ++r) {//iterates rows
            if (r > 0 && r % B == 0) os << "------+-------+------\n";//adds a divider between 3x3 block
rows
            for (int c = 0; c < N; ++c) {//iterates columns
                if (c > 0 && c % B == 0) os << "| ";//adds a divider between 3x3 block columns
                os << (isEmpty(r, c) ? '_' : static_cast<char>('0' + get(r, c)));//prints "_" for blanks
 and the number that's supposed to be there otherwise
                os << (c < N - 1 ? " " : "");//adds a space between cells
            }
            os << '\n';//adds a return line after each row
        }
    }

    void printRaw(std::ostream& os) const { //output grid in puzzle file format (no formatting)
        for (int r = 0; r < N; ++r) {
            for (int c = 0; c < N; ++c) {
                os << (isEmpty(r, c) ? ' ' : static cast<char>('0' + get(r, c)));
            }
            os << '\n';
        }
    }

    private:
        std::array<int, N * N> grid ;//where the board is stored
};

// ----------------------------------------------------------------------------
// SudokuSolver: exhaustive depth-first search with backtracking.
// Works on its own copy of the board and collects every solution found.
// ----------------------------------------------------------------------------
class SudokuSolver { //initializes class SudokuSolver responsible for the search, collects every
solution found
    public:
    //changed sodoku solver so now it takes the output stream and can immediately prints references
which are the solutions
    SudokuSolver(const SudokuBoard& puzzle, std::ostream& os, std::ostream& solutionFile, std::size_t
maxSolutions = 1000)// constructor, sets maximum solutions to 1000. This line and the next have been
changed and authored by me and Claude
        : board_(puzzle), os_(os), solutionFile_(solutionFile), maxSolutions_(maxSolutions) {} //copies
puzzleboard and stores the streams and limit
```

```
    std::size_t solveAll() {//method solveAll that returns # of solutions found, method has been changed
 and is authored by me
        count  = 0; //resets counter
        if (!board .givensAreConsistent()) return 0; //contradictory givens
        search(0); //start search at first cell
        return count ;//returns the number of solutions found
    }

    private:
        SudokuBoard board ;//copy of board
        std::ostream& os_; //added as a location to be streamed to. This and following three lines have
been authored by me
        std::ostream& solutionFile ; //output stream for solution file
        std::size t count  = 0; //added count
        std::size_t maxSolutions_; //added max amount of solutions, can be changed depending on if you
want more or less

        void search(int pos) {//method search that is the recursive step which searches for solutions
            if (maxSolutions_ != 0 && count_ >= maxSolutions_) { //max amount of solutions enforcement.
If statement authored by me
                return;//stops searching once max solutions have been reached
            }
            const int total = SudokuBoard::N * SudokuBoard::N;//total num of cells

            while (pos < total && !board .isEmpty(pos / SudokuBoard::N, //convert pos to row
                                                  pos % SudokuBoard::N)) //conver pos to col
                ++pos;//move to next cell

            // Every cell is filled: the solution is complete now and doesn't need to be stored
            if (pos == total) {// if solution is complete. If statement authored by me
                ++count ; //counter goes up one
                os_ << "\nSolution " << count_ << ":\n"; board_.print(os_); //prints solution without
storing anything
                board .printRaw(solutionFile ); //write solution to file in puzzle format
                return;//go back and find more solutions
            }

            int row = pos / SudokuBoard::N;//row of current empty cell
            int col = pos % SudokuBoard::N;//column of current empty cell

            for (int v = 1; v <= SudokuBoard::N; ++v) {//iterate 1-9
                if (board .canPlace(row, col, v)) { //if the digit is allowed there
                    board .set(row, col, v);//try the digit there
                    search(pos + 1);//recurse
                    board .set(row, col, SudokuBoard::EMPTY); //done recursing, go back
                }
            }
        }
};

class PuzzleRunner { //class PuzzleRunner that loads the file, solves the puzzle, and then prints
everything out
    public:
        explicit PuzzleRunner(const std::string& filename) : filename (filename) {} //stores file

        void run(std::ostream& os) {//method run that solve the puzzle and then prints it out
            os << "========================================\n";//seperator between each puzzle
            os << "Puzzle file: " << filename  << "\n\n";//prints name of puzzle file

            SudokuBoard puzzle;//empty board
            if (!puzzle.load(filename_)) {//if the puzzle cannot load
                os << "Error: could not read a valid 9x9 puzzle from this file.\n\n";//print that there
was an error
                return;//exit method
            }

            os << "Puzzle:\n";//prints "Puzzle:" with a return line
            puzzle.print(os);//prints the unsolved puzzle

            // Open solution file based on puzzle filename
```

```
            std::string solutionFilename = getSolutionFilename(filename );
            std::ofstream solutionFile(solutionFilename);

            SudokuSolver solver(puzzle, os, solutionFile); //creates an instance of Sudoku Solver called
 solver, updated to include solution file stream
            std::size t n = solver.solveAll();//calls solveAll which solves the puzzle and prints it

            os << '\n';//blank line
            if (n == 0) {//if there are no solutions
                os << "no solution found\n\n";//print that there aren't any solutions
                solutionFile << "No solution found.\n"; //write no solution message to file
                return;//exit method
            }
            else{ //if there are solutions
                os << n << (n==1 ? " solution" : " solutions") << " found.\n\n"; //print the number of
solutions, can now just print the amount of solutions instead of using a for loop
                solutionFile << "Solutions found: " << n << "\n"; //write solution count to file
            }
        }

    private:
        std::string filename ;//name of the file

        std::string getSolutionFilename(const std::string& puzzleFilename) const {
            // Extract puzzle number from filename (e.g., "puzzle1.txt" -> "solution1.txt")
            size t numStart = puzzleFilename.find last of("puzzle");
            if (numStart != std::string::npos && numStart + 1 < puzzleFilename.length()) {
                std::string numPart = puzzleFilename.substr(numStart + 1);
                size t dotPos = numPart.find('.');
                if (dotPos != std::string::npos) {
                    std::string num = numPart.substr(0, dotPos);
                    return "solution" + num + ".txt";
                }
            }
            return "solution.txt"; //fallback
        }
    };

    // ----------------------------------------------------------------------------
    int main(int argc, char* argv[]) {//main
        std::vector<std::string> files; //list of puzzle filenames
        if (argc > 1) { //if there are filenames given on command line
            for (int i = 1; i < argc; ++i) files.push back(argv[i]);//use each filename
        } else { //otherwise, use default puzzles
            for (int i = 1; i <= 5; ++i)//iterate 1-5
                files.push back("puzzle" + std::to string(i) + ".txt");//puzzle1.txt - puzzle5.txt
        }

        for (const std::string& f : files) {//iterate through puzzles
            PuzzleRunner runner(f); //create a PuzzleRunner runner for this file
            runner.run(std::cout); //solves puzzle and prints results
        }
        return 0;//exit program
}
```

GPT-6 Luna (`Codex/iterations/00-raw.cpp`)

```
//EECS 348 Assignment 4
//Sudoku Solver that use a brute force recursive algorithm
//input: puzzle1.txt through puzzle5.txt
//Output: solved puzzles
//Collaborators: Me
//Other Collaborators: Claude
//Weston G. Uptmor
//10/15/2026
//Revised by: ChatGPT, 10/28/2026

#include <array> //used for grid
#include <cctype> //checks whether a character is a digit
#include <fstream> //for reading files
#include <iostream> //for output
#include <iterator> //reads a whole file as a string
#include <string> //allows the use of strings
#include <vector> //allows a list of filenames

class SudokuBoard { //initializes class SudokuBoard responsible for the board and the rules of placement
    public:
        static const int N = 9;      // board is N x N
        static const int B = 3;      // block is B x B
        static const int EMPTY = 0;  // internal marker for a blank square

        SudokuBoard() { grid .fill(EMPTY); } //constructor that starts with every cell blank
        //following code reads puzzle from a file and returns it
        //code returns false if the file is invalid
        bool load(const std::string& filename) { //method "load"
            std::ifstream in(filename); //opens file
            if (!in) return false; //if file cannot be opened return false
            std::string text((std::istreambuf_iterator<char>(in)), //read entire file into one string
                             std::istreambuf_iterator<char>());
            int count = 0; //initializes the count of cells that have been read to 0
            for (char c : text) { //for loop that looks at each character in the file
                if (count >= N * N) { //if grid has been fully read
                    if (std::isdigit(static_cast<unsigned char>(c)) || c == '_') //if there are too many
 cells
                        return false;  // return false
                    continue; //ignore whitespace and other characters
                }
                if (c >= '1' && c <= '9')            grid_[count++] = c - '0'; //if a digit is already
given, convert it to an integer and store it
                else if (c == '_' || c == '.' || c == '0') grid_[count++] = EMPTY; //otherwise store it
as a blank space
                // all newlines and spaces are skipped
            }
            return count == N * N; //success if exactly 81 cells were read
        }

        int  get(int row, int col) const { return grid [row * N + col]; } //read a cell
        void set(int row, int col, int v) { grid [row * N + col] = v; } //write a value into said cell
        bool isEmpty(int row, int col) const { return get(row, col) == EMPTY; } //return true if cell is
 blank

        bool canPlace(int row, int col, int v) const { //initializes method canPlace that checks if num
can be place in a given spot
            for (int i = 0; i < N; ++i) { //for loop that iterates along a row and a col together
                if (get(row, i) == v) return false;//if v already appears in this row, return false
                if (get(i, col) == v) return false;//if v already appears in this collumn, return false
            }
            int br = (row / B) * B, bc = (col / B) * B; //top left corner of a cell's 3x3 block
            for (int r = br; r < br + B; ++r) //iterates over block's rows
                for (int c = bc; c < bc + B; ++c) //iterates over block's column's
                    if (get(r, c) == v) return false; //if v already appears in this block, return false
```

```
            return true;//if placement is good, return True
        }

        bool givensAreConsistent() const { //initializes method "givensAreConsistent" which checks if
the puzzle is unsolvable
            SudokuBoard tmp = *this; //creates a copy of the current board
            for (int r = 0; r < N; ++r) //iterates over rows
                for (int c = 0; c < N; ++c) { //iterates over columns
                    int v = tmp.get(r, c); //digit in the current cell
                    if (v == EMPTY) continue; //if the current cell is a blank, go to next one
                    tmp.set(r, c, EMPTY); //blanks out current cell
                    bool ok = tmp.canPlace(r, c, v); //checks if it would be legal to put current cell
back
                    tmp.set(r, c, v); //puts the digit back
                    if (!ok) return false; //if there is a conflict, returns false
                }
            return true; //if there were no conflicts, returns true
    }

    void print(std::ostream& os) const { //initializes method "print" which prints out the board
        for (int r = 0; r < N; ++r) {//iterates rows
            if (r > 0 && r % B == 0) os << "------+-------+------\n";//adds a divider between 3x3 block
rows
            for (int c = 0; c < N; ++c) {//iterates columns
                if (c > 0 && c % B == 0) os << "| ";//adds a divider between 3x3 block columns
                os << (isEmpty(r, c) ? '_' : static_cast<char>('0' + get(r, c)));//prints "_" for blanks
 and the number that's supposed to be there otherwise
                os << (c < N - 1 ? " " : "");//adds a space between cells
            }
            os << '\n';//adds a return line after each row
        }
    }

    private:
        std::array<int, N * N> grid ;//where the board is stored
};

// ----------------------------------------------------------------------------
// SudokuSolver: exhaustive depth-first search with backtracking.
// Works on its own copy of the board and collects every solution found.
// ----------------------------------------------------------------------------
class SudokuSolver { //initializes class SudokuSolver responsible for the search, collects every
solution found
    public:
    //changed sodoku solver so now it takes the output stream and can immediately prints references
which are the solutions
    SudokuSolver(const SudokuBoard& puzzle, std::ostream& os)// constructor
        : board (puzzle), os (os) {} //copies puzzleboard and stores the output stream

    std::size_t solveAll() {//method solveAll that returns # of solutions found, method has been changed
 and is authored by me
        count_ = 0; //resets counter
        if (!board .givensAreConsistent()) return 0; //contradictory givens
        search(0); //start search at first cell
        return count ;//returns the number of solutions found
    }

    private:
        SudokuBoard board ;//copy of board
        std::ostream& os_; //added as a location to be streamed to. This and following two lines have
been authored by me
        std::size_t count_ = 0; //added count

        void search(int pos) {//method search that is the recursive step which searches for solutions
            const int total = SudokuBoard::N * SudokuBoard::N;//total num of cells

            while (pos < total && !board .isEmpty(pos / SudokuBoard::N, //convert pos to row
                                                  pos % SudokuBoard::N)) //conver pos to col
```

```
                ++pos;//move to next cell

            // Every cell is filled: the solution is complete now and doesn't need to be stored
            if (pos == total) {// if solution is complete. If statement authored by me
                ++count ; //counter goes up one
                board .print(os ); //prints each solution in the input grid format
                return;//go back and find more solutions
            }

            int row = pos / SudokuBoard::N;//row of current empty cell
            int col = pos % SudokuBoard::N;//column of current empty cell

            for (int v = 1; v <= SudokuBoard::N; ++v) {//iterate 1-9
                if (board .canPlace(row, col, v)) { //if the digit is allowed there
                    board .set(row, col, v);//try the digit there
                    search(pos + 1);//recurse
                    board .set(row, col, SudokuBoard::EMPTY); //done recursing, go back
                }
            }
        }
};

class PuzzleRunner { //class PuzzleRunner that loads the file, solves the puzzle, and then prints
everything out
    public:
        explicit PuzzleRunner(const std::string& filename) : filename (filename) {} //stores file

        void run() {//loads and solves one puzzle, writing the required result file
            std::string base = filename ;
            std::size t slash = base.find last of("/\\");
            if (slash != std::string::npos) base = base.substr(slash + 1);
            std::string number;
            for (char c : base) if (c >= '0' && c <= '9') number += c;
            std::string outputName = "solution" + (number.empty() ? base : number) + ".txt";
            std::ofstream os(outputName);
            if (!os) return;

            SudokuBoard puzzle;
            if (!puzzle.load(filename )) {
                os << "No solution found.\n";
                return;
            }

            SudokuSolver solver(puzzle, os);
            std::size t n = solver.solveAll();
            if (n == 0) os << "No solution found.\n";
            else os << "Solutions found: " << n << '\n';
        }

    private:
        std::string filename ;//name of the file
    };

    // -----------------------------------------------------------------------------
    int main(int argc, char* argv[]) {//main
        std::vector<std::string> files; //list of puzzle filenames
        if (argc > 1) { //if there are filenames given on command line
            for (int i = 1; i < argc; ++i) files.push back(argv[i]);//use each filename
        } else { //otherwise, use default puzzles
            for (int i = 1; i <= 5; ++i)//iterate 1-5
                files.push back("puzzle" + std::to string(i) + ".txt");//puzzle1.txt - puzzle5.txt
        }

        for (const std::string& f : files) {//iterate through puzzles
            PuzzleRunner runner(f); //create a PuzzleRunner runner for this file
            runner.run(); //solves puzzle and writes its result file
        }
        return 0;//exit program
}
```